FP8 Training and FP8 Activation Compression

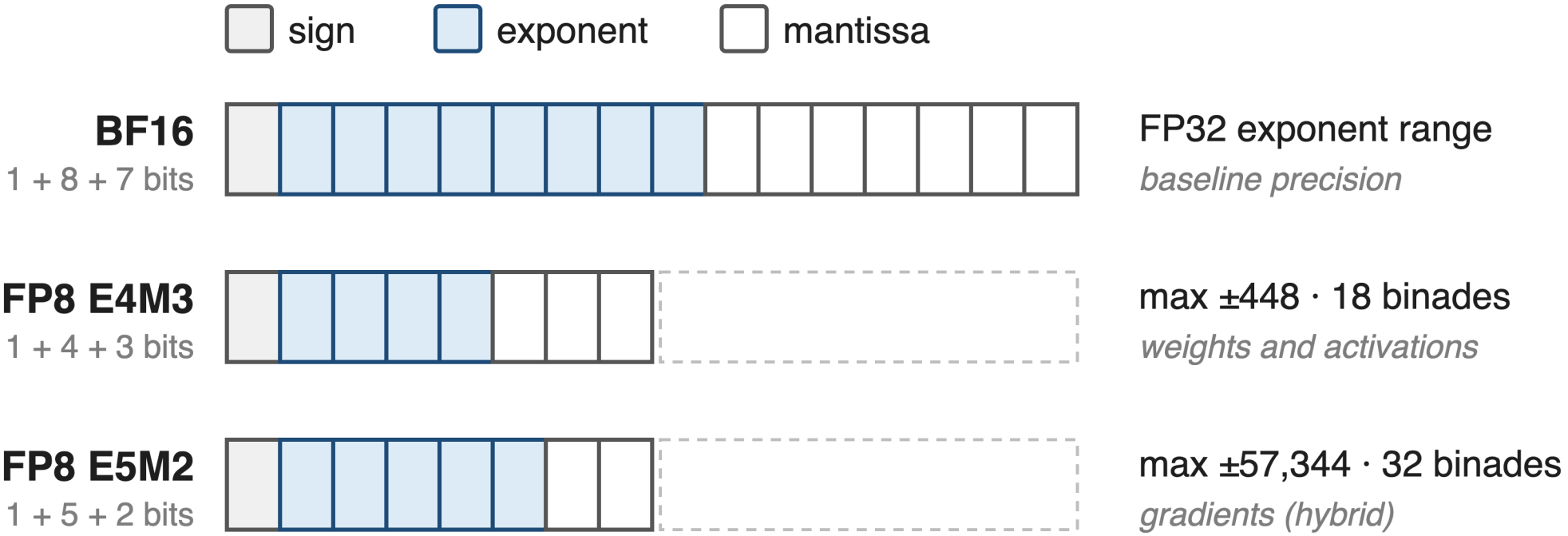
Per-tensor and blockwise FP8 GEMMs, FP8-cached activations

NVIDIA Transformer Engine · Megatron-Core · DeepSeek-V3 · Kimi K2

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Low-Precision & Quantization · github.com/NVIDIA/TransformerEngine

Hopper pretraining runs its GEMMs in 8-bit floats



DeepSeek-V3 (671B MoE, 14.8T tokens) ran every linear-layer GEMM in FP8 on H800s

BF16 training leaves the FP8 tensor cores idle

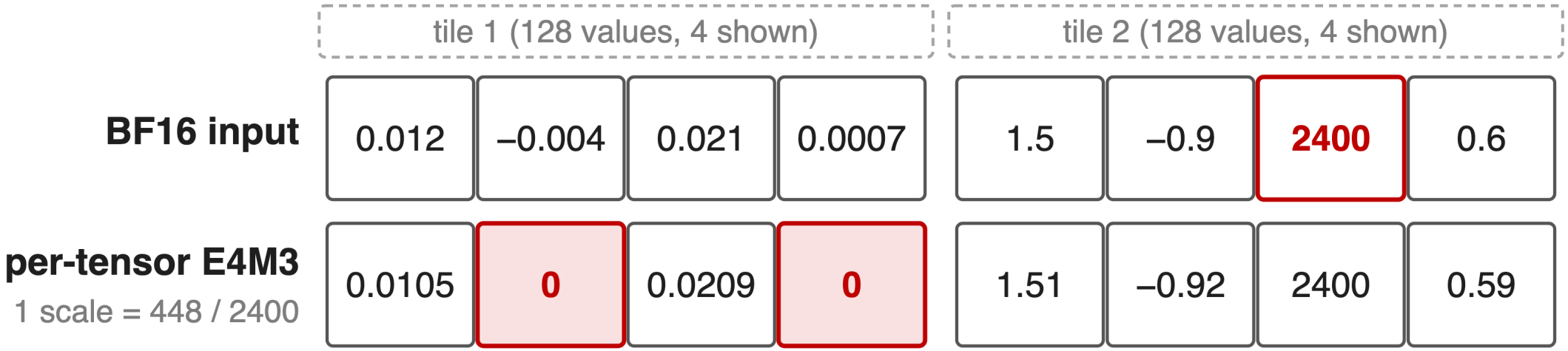
1.92×

raw FP8 GEMM speed over BF16 on H200, 5B-model shapes

- The catch: E4M3 spans only **18 binades** per scale

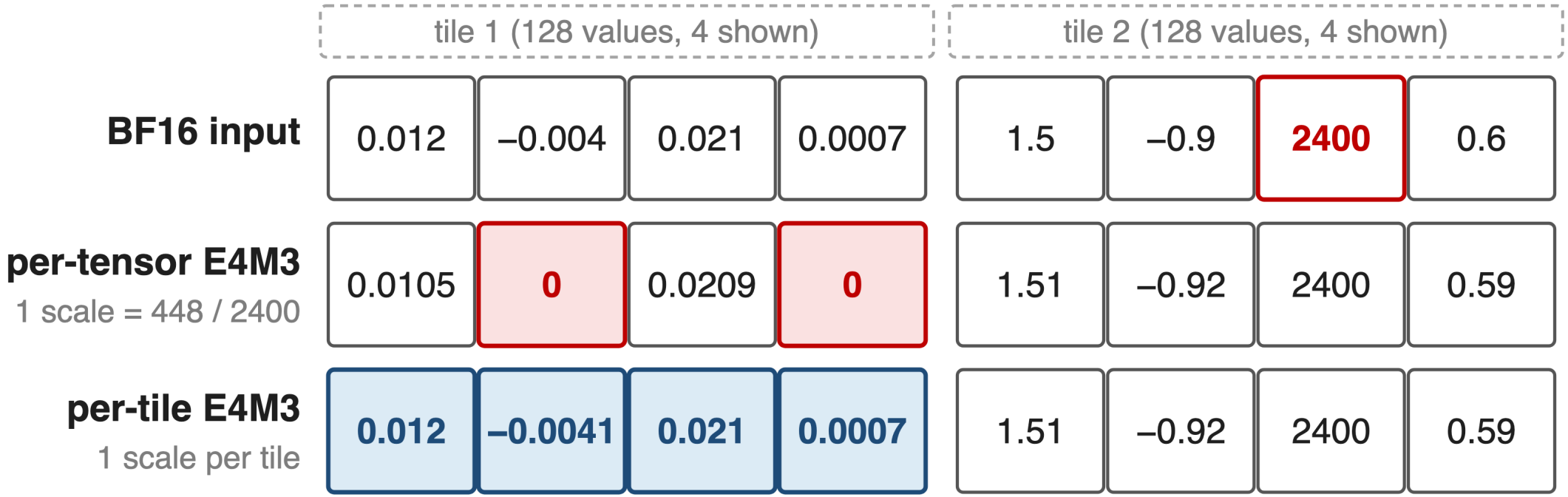
Transformer Engine docs, "GEMM Speedups Across Precisions": 5B model on H200 NVL, pre-quantized inputs

One scale per tensor lets outliers erase values



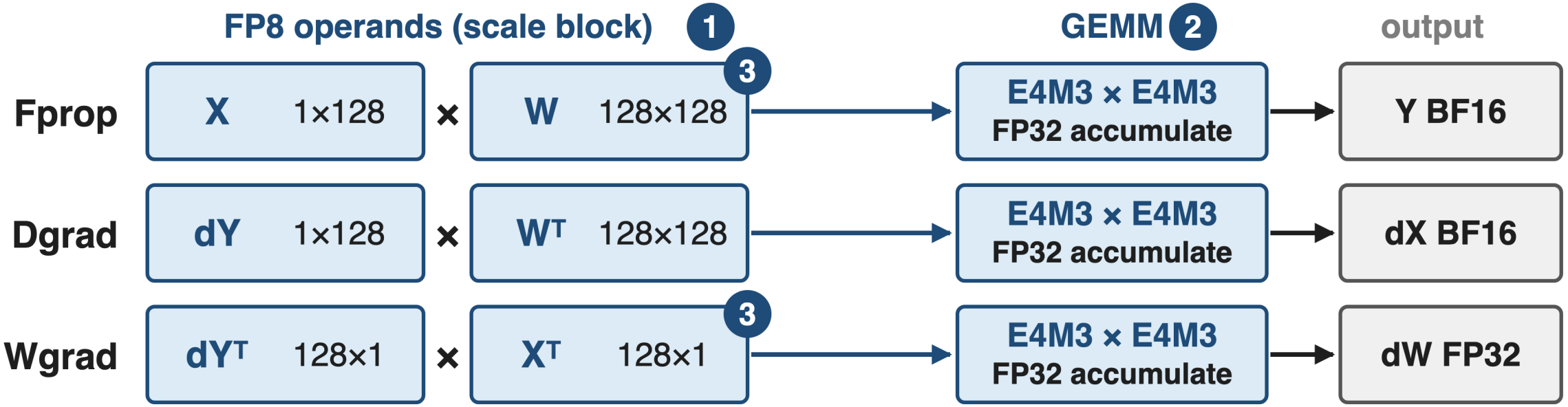
One outlier (2400) sets the scale; small values fall below E4M3's 2^{-9} floor and flush to 0

Insight: outliers are local, so scales must be local



Give every 128 values their own scale, and E4M3 holds activations, weights and gradients

Each linear layer runs three GEMMs in FP8



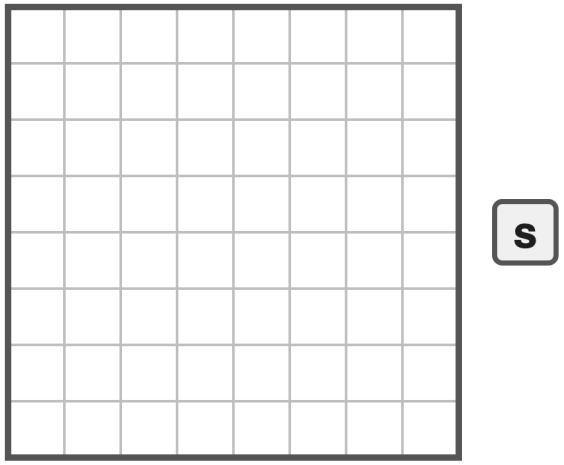
kept in BF16 / FP32

Embedding | LM head | Router FP32 | Norms | Attention | Optimizer FP32

❶ scale blocks · ❷ FP32 accumulation · ❸ FP8 caching and FP8 all-gather

❶ Scale each 1×128 tile and 128×128 block

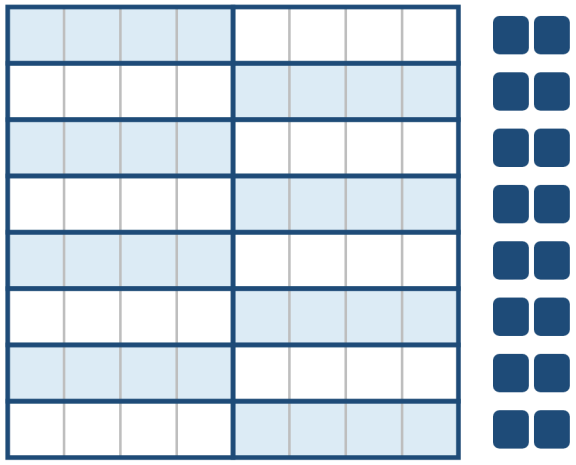
Per-tensor



1 FP32 scale

delayed or current amax

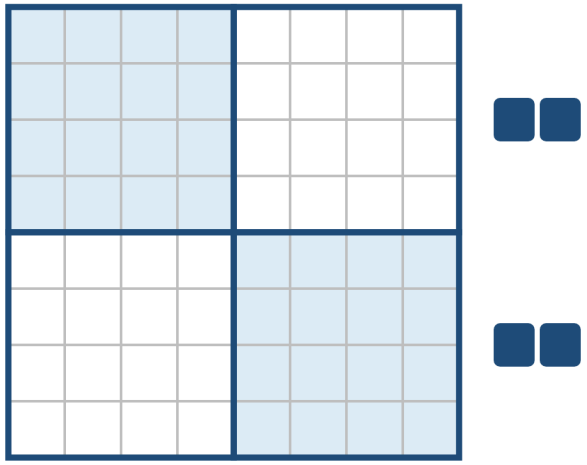
1×128 tiles



1 scale per tile

activations and gradients

128×128 blocks

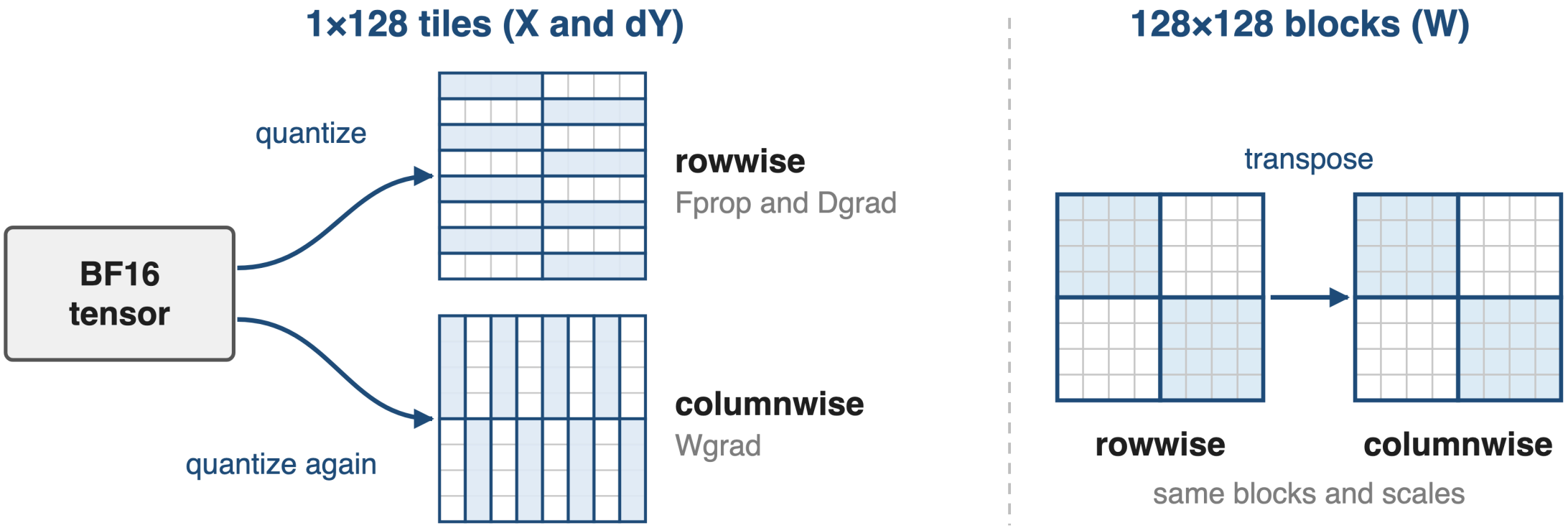


1 scale per block

weights

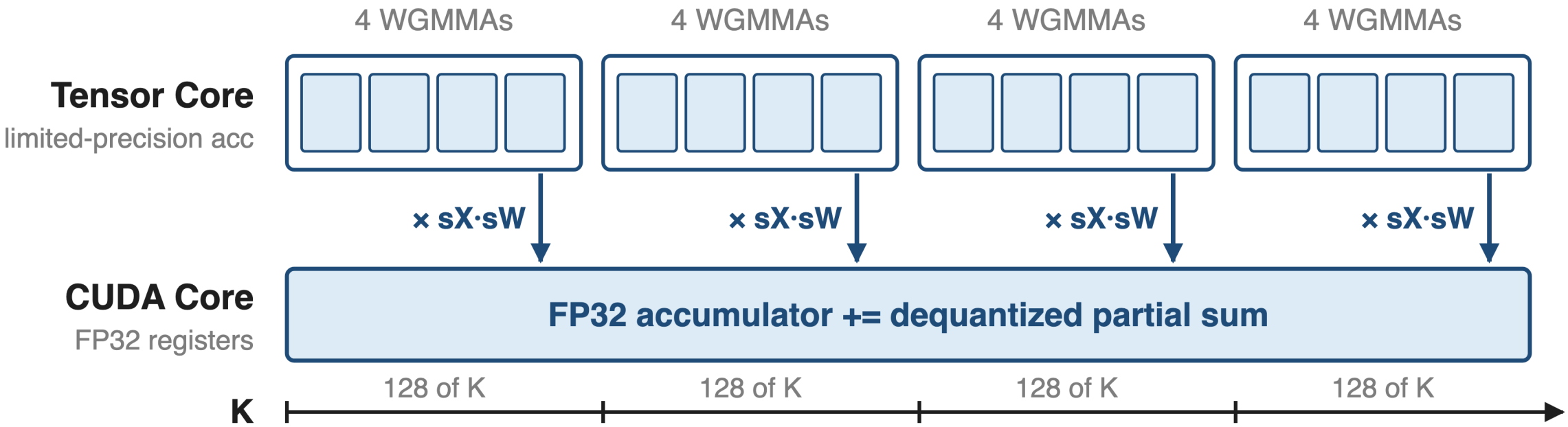
Scale = 448 / amax of the block, rounded down to a power of 2, kept in FP32

❶ Row tiles must be quantized twice



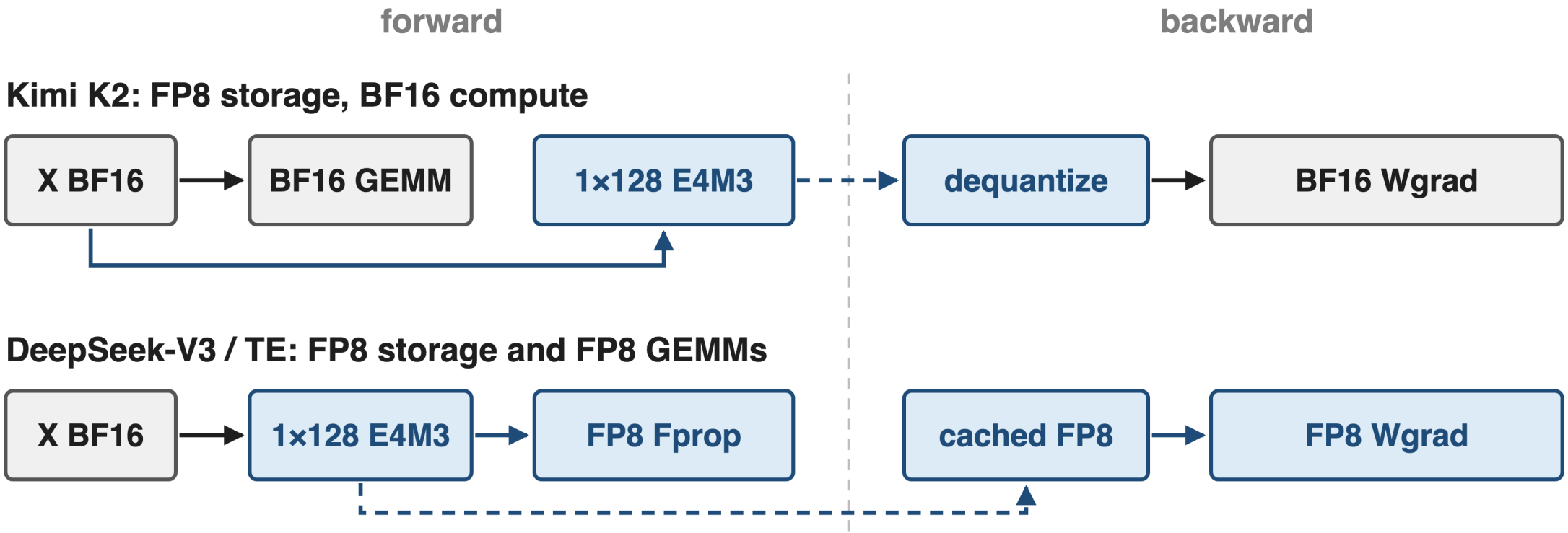
Why not 128×128 everywhere? Block-scaled activation gradients diverged a 16B MoE at ~300B tokens

❷ Accumulate in FP32 every 128 elements



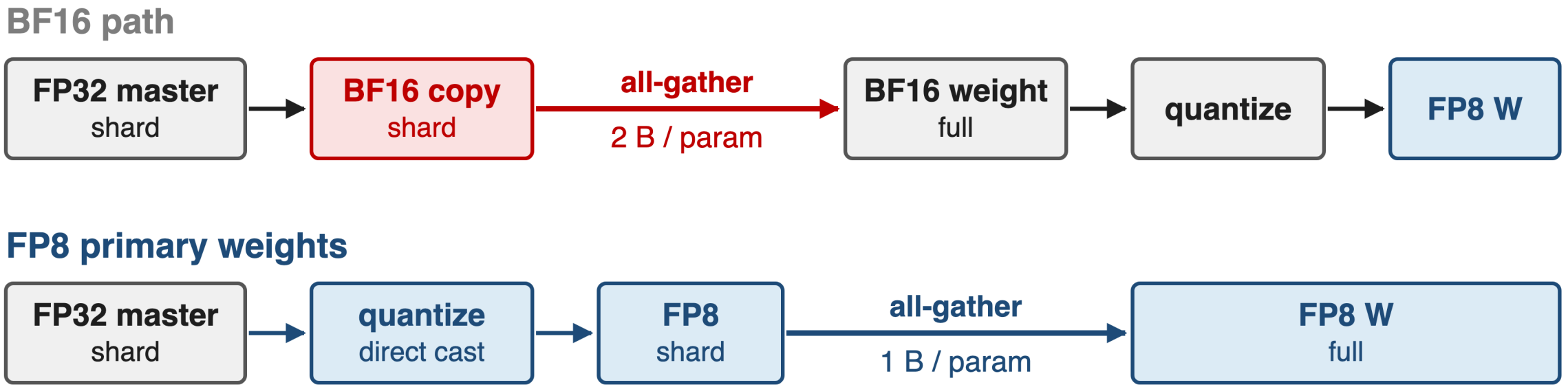
H800 FP8 tensor cores keep ~14 accumulator bits: nearly 2% max error at K = 4096

❸ Cache activations in FP8, even for BF16 GEMMs



Kimi K2: MoE up-projection and SwiGLU inputs as 1×128 E4M3 tiles, no measurable loss increase

❸ Cast FP32 masters straight to FP8 for all-gather



Half the all-gather bytes and no BF16 weight copy; MXFP8 cannot do this (deck 47)

Blockwise FP8 stays within 0.25% of BF16 loss

< 0.25%

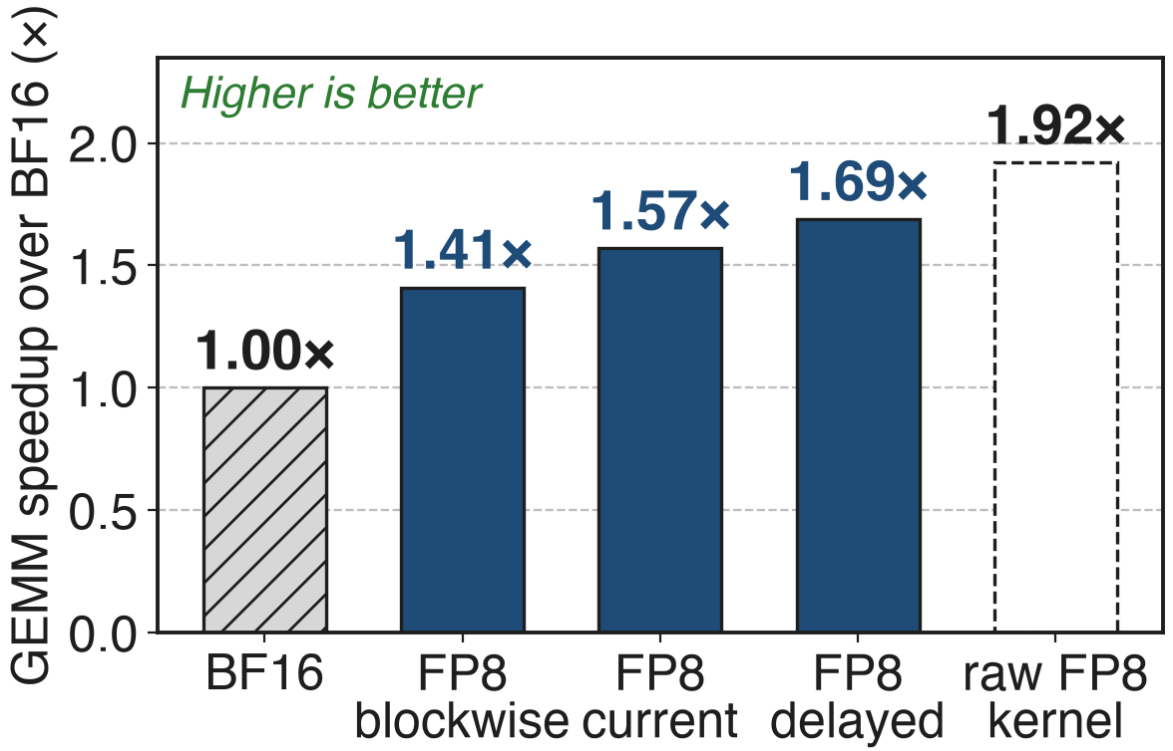
relative loss error vs BF16: 16B MoE (1.33T tokens), 230B MoE (0.9T tokens)

- Per-tensor FP8 (simulated): GPT-175B perplexity **6.68** vs **6.65**

DeepSeek-V3 Technical Report, Sec. 3.3 and App. B.1 (1×128 / 128×128 scales + FP32 promotion)

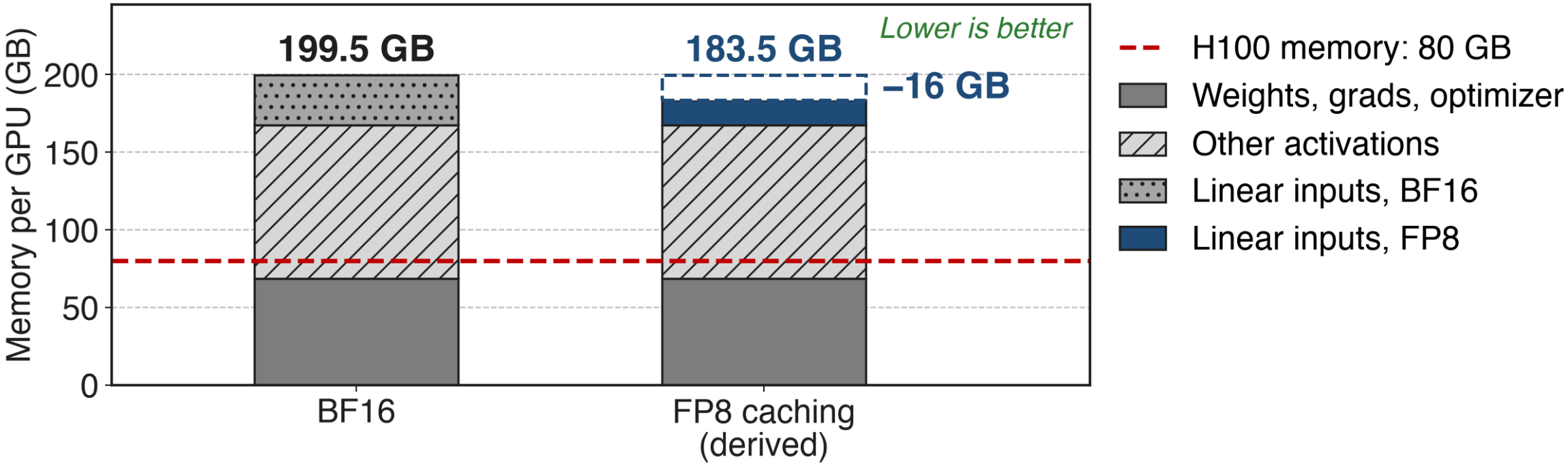
FP8 GEMMs run 1.4–1.7× faster than BF16 on H200

- Includes per-step quantization (5B model)
- Blockwise is slowest: Hopper emulates block scales
- End to end: **~10–25%** for large MoE



At scale: DeepSeek-V3 on 1,024 H100s reaches 368 TFLOPS/GPU with blockwise FP8

FP8 caching saves ~16 GB of activations per GPU



Only ~32 of 131 GB are FP8-eligible; pair with recompute (deck 52) and offload (deck 54)

The price: emulated scales, copies, padding

- Hopper emulates block scales: **1.41×** vs **1.69×** per-tensor
- Two FP8 copies per 1×128 tensor; BF16-weight layers save no memory
- Shapes must divide by 128; MoE pads tokens per expert

On Blackwell, MXFP8 (deck 47) makes block scaling native; NVFP4 (deck 48) goes to 4 bits

Summary

- BF16 leaves **1.92×** raw FP8 GEMM speed unused on H200
- **Insight**: outliers are local, so scale per 1×128 tile / 128×128 block and accumulate in FP32
- **< 0.25%** loss gap; **1.41–1.69×** GEMM speedup; **~16 GB** activations saved
- Next rungs: MXFP8 (deck 47), NVFP4 (deck 48)

https://github.com/NVIDIA/TransformerEngine